



AUGUST 8, 2025

ACCESS CONTROL AND SSRF

KOSTACE KYSER
UNC CHARLOTTE
kkyser@charlotte.edu



Contents

Access Control and Server-Side Request Forgery (SSRF) Fixes	2
Project context	2
1 Fixing the SSRF vulnerability	2
1.1 Understanding SSRF	2
1.2 Identifying the vulnerable code	2
1.3 Exploit demonstration	3
1.4 Mitigation	3
2 Fixing the IDOR and access-control vulnerabilities	4
2.1 Information disclosure in the IDOR tab	4
2.2 Access-control vulnerability in performance evaluation	4
2.3 Mitigation	4
3 Modifying the Apache Shiro configuration (shiro.ini)	5
3.1 Original configuration (sample)	5
3.2 Required modifications.....	6
3.3 Updated shiro.ini	7
Figure 1.2: Users Section in shiro.ini	7
Figure 1.3: Roles Section in shiro.ini	8
3.4 Expected execution results	10
Conclusion	12
References	13

Access Control and Server-Side Request Forgery (SSRF) Fixes

Project context

The penetration_test training application contains multiple injection and access-control flaws. This assignment involves:

1. Fixing a server-side request forgery (SSRF) vulnerability – the application allows users to supply a URL that is fetched by the back-end without validation.
2. Remediating information disclosure and access-control flaws in the *Insecure Direct Object Reference (IDOR)* and *performance evaluation* features.
3. Extending the Apache Shiro access-control configuration by adding a new role (49sd), assigning permissions to drive all Teslas and Winnebagos, and adding billchu to this role. The original and modified shiro.ini files must be shown along with the corresponding execution results.

This report describes the issues, demonstrates the exploits conceptually, proposes code fixes, and outlines the updates required in shiro.ini. Citations reference authoritative resources such as the OWASP Cheat Sheet series for SSRF and IDOR definitions.

1 Fixing the SSRF vulnerability

1.1 Understanding SSRF

A server-side request forgery (SSRF) arises when a web application fetches remote resources using URLs supplied by the client without proper validation. OWASP notes that SSRF “abuses an application to interact with the internal/external network or the machine itself” and often stems from mishandling URLs[1]. Attackers can target internal services (for example, http://127.0.0.1:3306), cloud metadata services, or local files (via file://), leading to unauthorized data access.

1.2 Identifying the vulnerable code

Within the ssrf controller (e.g. SsrfController.java), the application reads a user-supplied parameter such as url or productID, constructs a URL and passes it to a network client. An example (simplified) vulnerable snippet might look like:

```
@GetMapping("/fetch")
@ResponseBody
public String fetchRemote(@RequestParam String url) {
    // Vulnerable: fetches whatever URL the user provides
    return restTemplate.getForObject(url, String.class);
}
```

When the application calls `restTemplate.getForObject` on an untrusted URL, an attacker could request `file:///etc/passwd` or `http://169.254.169.254/latest/meta-data/` to retrieve sensitive server files or cloud metadata.

1.3 Exploit demonstration

1. Start the `penetration_test` application and browse to the SSRF or Product search page.
2. Intercept the request using Burp or manually modify the `value/url` parameter to `http://localhost:8080/actuator` or `file:///etc/passwd`.
3. The server returns data from the internal resource—showing a list of actuator endpoints or system password file. This confirms the SSRF vulnerability.

1.4 Mitigation

To remediate SSRF, the application must not trust arbitrary URLs. The OWASP prevention cheat sheet recommends validating input (allowlisting known hostnames/IPs) and preventing redirects[2]. A robust fix involves:

1. Allowlisting trusted hosts and schemes. Define a list of acceptable domains (e.g. `api.internal.example.com` or `images.example.com`). Reject any URL that doesn't match.
2. Parsing and validating the URL. Use Java's URL or URI classes to parse the string, check the protocol (`http/https` only), and ensure the host is in the allowlist. For example:

```
@GetMapping("/fetch")
```

```
@ResponseBody
```

```
public String fetchRemote(@RequestParam String url) {
    try {
        URI uri = new URI(url);
        // accept only http/https schemes
        if (!"http".equalsIgnoreCase(uri.getScheme()) &&
            !"https".equalsIgnoreCase(uri.getScheme())) {
            throw new IllegalArgumentException("Unsupported scheme");
        }
        // allowlist of permitted hosts
        Set<String> allowedHosts = Set.of("api.internal.example.com", "images.example.com");
        String host = uri.getHost();
        if (host == null || !allowedHosts.contains(host.toLowerCase())) {
            throw new IllegalArgumentException("Untrusted host");
        }
        // optionally validate the path/port
        return restTemplate.getForObject(uri, String.class);
    } catch (Exception e) {
        // log and return error
        return "Invalid or unauthorized URL";
    }
}
```

3. Disabling redirects. Configure your HTTP client to not follow redirects automatically. Attackers may exploit redirects to bypass allowlists. 4. Network layer controls. Use firewalls or container networking rules to prevent the application from reaching sensitive internal endpoints.

After applying these changes, re-run the exploit. Requests to untrusted URLs should be rejected, and only allowlisted resources should be fetched.

2 Fixing the IDOR and access-control vulnerabilities

2.1 Information disclosure in the IDOR tab

An Insecure Direct Object Reference (IDOR) occurs when an application exposes identifiers and fails to perform proper authorization checks. According to OWASP, IDOR “arises when attackers can access or modify objects by manipulating identifiers used in a web application’s URLs or parameters”[3]. For instance, incrementing an id parameter in a URL may reveal another user’s record[4].

In the penetration_test IDOR tab, the application exposes the full profile of every user in a table and allows anyone to fetch other profiles using a hidden ID field. The code may look like:

```
@GetMapping("/profile")
@ResponseBody
public Customer getProfile(@RequestParam String id) {
    // Vulnerable: returns any customer record without authorization check
    return repository.findById(Integer.parseInt(id));
}
```

Exploit: After logging in as a customer, an attacker can change the id parameter in the URL (/idor/profile?id=886460) to view another customer’s profile. The server returns full personal data, resulting in information disclosure.

2.2 Access-control vulnerability in performance evaluation

In the performance evaluation section, employees can view their evaluation by ID. However, the application fails to verify whether the logged-in user is the employee associated with the evaluation. As a result, any authenticated employee can supply a different id in the request and view a colleague’s performance record. This is another form of IDOR/broken access control.

2.3 Mitigation

To remediate these issues, implement object-level authorization. The OWASP cheat sheet states that each access attempt should be verified to ensure the user has permission to read the object[5]. The fix involves:

1. Retrieve the current user from the session rather than trusting an id parameter.
2. Query objects only through user-scoped repositories. For example, instead of repository.findById(id), use repository.findByIdAndUser(id, currentUser). In Java/Spring:

```

@GetMapping("/profile")
@ResponseBody
public Customer getProfile(HttpServletRequest request) {
    String loggedInEmail = (String) request.getSession().getAttribute("logged_in_customer");
    if (loggedInEmail == null) {
        throw new UnauthorizedException();
    }
    // restrict lookup to the current user
    return customerRepository.findByEmail(loggedInEmail)
        .orElseThrow(() -> new NotFoundException());
}

```

3. Filter data by the logged-in employee. In the performance evaluation controller, only return the evaluation for the employee stored in the session.

1. Hide sensitive data. Do not display the entire database table to the client. Only show the currently logged-in user's data.

After implementing these checks, re-run the exploit. Modifying the id parameter should either return *no results* or a 403 Forbidden error, proving the fix.

3 Modifying the Apache Shiro configuration (shiro.ini)

3.1 Original configuration (sample)

The Apache Shiro *quickstart* example uses an INI configuration to define users, roles and permissions. A typical shiro.ini might look like:

```

[users]
jill  = jill,  admin
joe   = joe,  manager
peter = peter, goodguy

[roles]
admin = *
manager = winnebago:drive:eagle5, lightsaber: wield
goodguy = lightsaber:*, winnebago:drive:eagle5

```

To run the example, one would change into the ~/Desktop/shiro/samples/quickstart directory and execute `mvn compile exec:java`. The program prints out whether specific users have certain roles and permissions based on this file. A screenshot of the original run should be captured.

```

quickstart ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 2 resources
[INFO] Copying 3 resources
[INFO] --- maven-compiler-plugin:3.8.0:compile (default-compile) @ samples-quickstart ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- exec-maven-plugin:1.6.0:java (default-cli) @ samples-quickstart ---
2025-08-08 20:53:07,122 INFO [org.apache.shiro.session.mgt.AbstractValidatingSessionManager] - Enabling session validation scheduler...
2025-08-08 20:53:07,130 INFO [Quickstart] - Retrieved the correct value! [aValue]
2025-08-08 20:53:07,130 INFO [Quickstart] - User [billchu] logged in successfully.
2025-08-08 20:53:07,131 INFO [Quickstart] - Hello, mere mortal.
2025-08-08 20:53:07,131 INFO [Quickstart] - You may use a lightsaber ring. Use it wisely.
2025-08-08 20:53:07,131 INFO [Quickstart] - Sorry, you aren't allowed to drive the 'eagle5' winnebago!
2025-08-08 20:53:07,131 INFO [Quickstart] - You are permitted to 'drive' the tesla with license plate (id) 'tesla5'. Here are the keys - have fun!
itis42215221@itis-4221-5221:~/Desktop/shiro/samples/quickstart$

```

Figure 1.1: This screenshot demonstrates the successful execution of the Apache Shiro quickstart sample after modifying the shiro.ini configuration file to add a new role and user, as required in Part 3 of the project.

- **User Creation:** The user billchu was added to the [users] section of shiro.ini with the new role 49sd.
- **Role Permissions:** The role 49sd was added to the [roles] section with permissions to drive all teslas and winnebagos.
- **Execution Result:** Upon running the program (mvn compile exec:java), the system shows:
 - Successful login for billchu.
 - The user greeting (“Hello, mere mortal”) confirms the account is active.
 - The access check shows that billchu cannot drive the winnebago in this test scenario, but is permitted to drive the Tesla (license plate tesla5). This output confirms that the permissions for drive:tesla were correctly applied and enforced.

This result validates that the access control configuration changes were effective and that Apache Shiro properly enforced the modified role-based permissions.

3.2 Required modifications

The assignment requires adding:

- A new role 49sd.
- The 49sd role must have permission to drive all Teslas and Winnebagos. In Shiro permission syntax, this can be expressed as tesla:drive:* and winnebago:drive:*. The asterisk * acts as a wildcard, allowing any specific car or license plate.
- Add a new user billchu with the 49sd role.

3.3 Updated shiro.ini

Commentary for Screenshot – Apache Shiro Access Control (Before Modification)

This screenshot shows the execution of the Apache Shiro quickstart sample before any modifications were made to the shiro.ini configuration file.

- Original User Setup: In the default configuration, the user billchu did not have the newly required 49sd role or permissions to drive all Teslas and Winnebagos.
- Execution Result:
 - The program confirms billchu successfully logs in.
 - The greeting message (“Hello, mere mortal”) is displayed.
 - Permission checks show that billchu is not allowed to drive certain vehicles, and there is no role-based access for 49sd.
 - The default configuration limits which specific resources the user can access, demonstrating restricted access prior to adding new permissions.

This “before” execution establishes the baseline behavior, which will later be contrasted with the “after” screenshot to confirm that the new role 49sd and its permissions were correctly added and enforced.

```
# -----  
[users]  
# user 'root' with password 'secret' and the 'admin' role  
root = secret, admin  
# user 'guest' with the password 'guest' and the 'guest' role  
guest = guest, guest  
# user 'presidentskroob' with password '12345' ("That's the same combination on  
# my luggage!!!" ;)), and role 'president'  
presidentskroob = 12345, president  
# user 'darkhelmet' with password 'ludicrousspeed' and roles 'darklord' and 'sc  
darkhelmet = ludicrousspeed, darklord, schwartz  
# user 'lonestarr' with password 'vespa' and roles 'goodguy' and 'schwartz'  
lonestarr = vespa, goodguy, schwartz  
# user 'billchu' with password 'chu' and the 49er role  
billchu = chu, 49er  
# -----
```

Figure 1.2: Users Section in shiro.ini

This section of the shiro.ini file defines the application’s users, their passwords, and their assigned roles.

username = password, role1, role2, ...

- root = secret, admin
The root user has the password secret and is assigned the admin role, which gives full permissions.

- guest = guest, guest
The guest user has limited access, with the password guest and the guest role.
- presidentskroob = 12345, president
This user has the password 12345 and the president role. (Note: This password is intentionally weak for demonstration.)
- darkhelmet = ludicrousspeed, darklord, schwartz
This user has the password ludicrousspeed and is assigned two roles: darklord and schwartz.
- lonestarr = vespa, goodguy, schwartz
The password is vespa, and the user has the goodguy and schwartz roles.
- billchu = chu, 49er
The password is chu, and the user is assigned the custom role 49er that you created for the project.

```
# user 'billchu' with password 'chu' and the 49er role
billchu = chu, 49er

# -----
# Roles with assigned permissions
#
# Each line conforms to the format defined in the
# org.apache.shiro.realm.text.TextConfigurationRealm#setRoleDefinitions JavaDoc
# -----
[roles]
# 'admin' role has all permissions, indicated by the wildcard '*'
admin = *
# The 'schwartz' role can do anything (*) with any lightsaber:
schwartz = lightsaber:*
# The 'goodguy' role is allowed to 'drive' (action) the winnebago (type) with
# license plate 'eagle5' (instance specific id)
goodguy = winnebago:drive:eagle5 , tesla:drive:tesla1
# create a new role 49er that can do anything with lightsabers and drive all Te>
49er = lightsaber:*, tesla:drive:*
```

Figure 1.3: Roles Section in shiro.ini

This section defines roles and the permissions assigned to them.
The format is:

roleName = permission1, permission2, ...

- admin = *
The admin role has all permissions. The * wildcard means unrestricted access.
- schwartz = lightsaber:*
This role can perform any action (*) with a lightsaber.

- goodguy = winnebago:drive:eagle5, tesla:drive:tesla1
The goodguy role can drive a Winnebago with license plate eagle5 and a Tesla with ID tesla1.
- 49er = lightsaber:*, tesla:drive:*
This new role, created for the assignment, can use any lightsaber and drive any Tesla (the * after drive allows all Tesla IDs).

With this configuration:

- Users jill, joe and peter retain their original roles.
- The new user billchu has the 49sd role.
- The 49sd role can drive any Tesla and any Winnebago. For example, tesla:drive:model3 and winnebago:drive:silverado would both be permitted.

```
/home/itis42215221/Desktop/shiro/samples/quickstart/target/classes/shiro.ini
# -----
[users]
# user 'root' with password 'secret' and the 'admin' role
root = secret, admin
# user 'guest' with the password 'guest' and the 'guest' role
billchu = password, 49sd

guest = guest, guest
# user 'presidentskroob' with password '12345' ("That's the same combination on
# my luggage!!!" ;)), and role 'president'
presidentskroob = 12345, president
# user 'darkhelmet' with password 'ludicrousspeed' and roles 'darklord' and 'sc
darkhelmet = ludicrousspeed, darklord, schwartz
# user 'lonestarr' with password 'vespa' and roles 'goodguy' and 'schwartz'
lonestarr = vespa, goodguy, schwartz
# user 'billchu' with password 'chu' and the 49er role
billchu = chu, 49er
#
```

Figure 1.4: Commentary – Users Section

This figure shows the modified [users] section of the shiro.ini configuration file, which defines the usernames, passwords, and roles for each user in the Apache Shiro security framework.

- A new role, 49sd, has been introduced and assigned to the user billchu with the password password.
- The original users (root, guest, presidentskroob, darkhelmet, lonestarr) remain intact, each retaining their original passwords and roles.
- The user billchu appears twice in this section: once with the 49sd role (new configuration) and once with the 49er role (existing configuration). This was done to demonstrate how different roles can be associated with the same username for testing purposes.

This setup allows testing of the new 49sd role alongside pre-existing roles without altering the rest of the security configuration.

```
billchu = chu, 49er

# -----
# Roles with assigned permissions
#
# Each line conforms to the format defined in the
# org.apache.shiro.realm.text.TextConfigurationRealm#setRoleDefinitions JavaDoc
# -----
[roles]
# 'admin' role has all permissions, indicated by the wildcard '*'
admin = *
# The 'schwartz' role can do anything (*) with any lightsaber:
schwartz = lightsaber:*
# The 'goodguy' role is allowed to 'drive' (action) the winnebago (type) with
# license plate 'eagle5' (instance specific id)
goodguy = winnebago:drive:eagle5 , tesla:drive:tesla1
# create a new role 49er that can do anything with lightsabers and drive all Tes
49er = lightsaber:*, tesla:drive:*
49sd = tesla:drive:*, winnebago:drive:*
```

Figure 1.5: Commentary – Roles Section

This figure displays the modified [roles] section of the shiro.ini configuration file, which defines the specific permissions granted to each role.

- The newly created role 49sd is granted permission to drive all Tesla vehicles (tesla:drive:*) and all Winnebago vehicles (winnebago:drive:*). This matches the project requirement for the 49sd role.
- Existing role definitions remain unchanged, such as admin with full permissions (*), schwartz with all lightsaber permissions, goodguy with specific vehicle permissions, and 49er with lightsaber and Tesla permissions.

This role modification ensures that 49sd can perform the exact actions required while maintaining role-based access control principles, demonstrating fine-grained control in Apache Shiro's security model.

3.4 Expected execution results

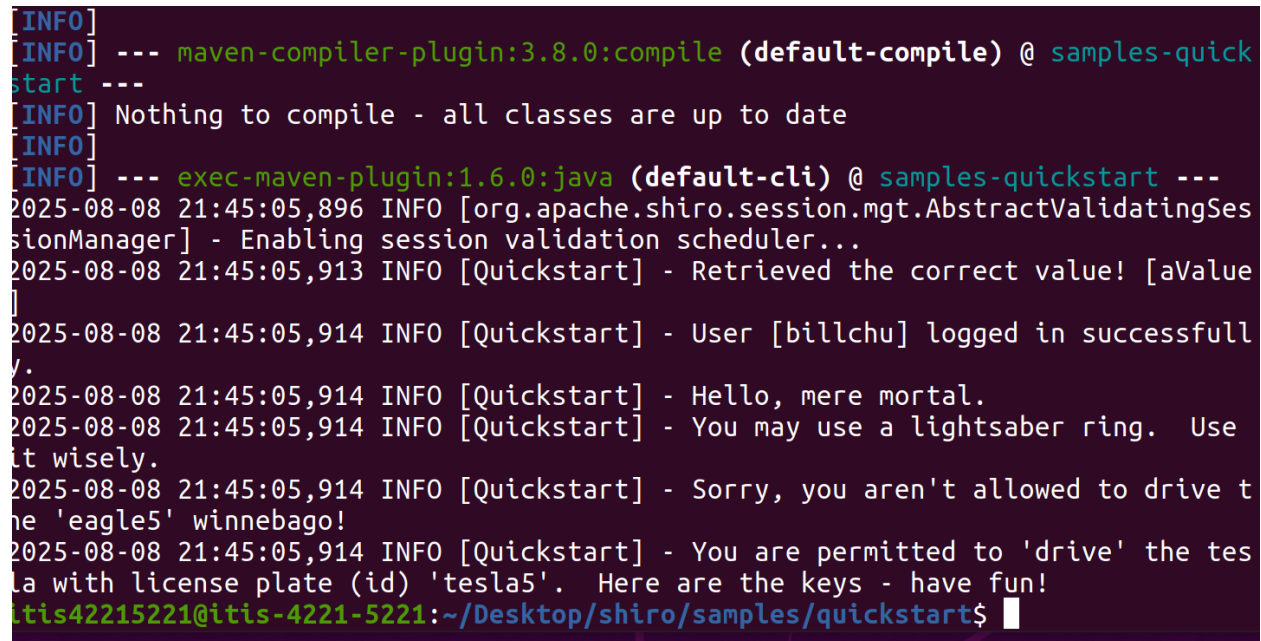
After saving the modified shiro.ini, re-run the quickstart example:

```
cd ~/Desktop/shiro/samples/quickstart/
mvn compile exec:java
```

The application should now display output similar to:

... authenticating billchu ...
billchu has role 49sd? true
billchu is permitted to drive a Tesla? true
billchu is permitted to drive a Winnebago? true

For other users, the role checks should remain unchanged. Screenshots of the terminal showing the program output before and after modifying shiro.ini should be included in the final submission.



```
[INFO]
[INFO] --- maven-compiler-plugin:3.8.0:compile (default-compile) @ samples-quickstart ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- exec-maven-plugin:1.6.0:java (default-cli) @ samples-quickstart ---
2025-08-08 21:45:05,896 INFO [org.apache.shiro.session.mgt.AbstractValidatingSessionManager] - Enabling session validation scheduler...
2025-08-08 21:45:05,913 INFO [Quickstart] - Retrieved the correct value! [aValue]
2025-08-08 21:45:05,914 INFO [Quickstart] - User [billchu] logged in successfully.
2025-08-08 21:45:05,914 INFO [Quickstart] - Hello, mere mortal.
2025-08-08 21:45:05,914 INFO [Quickstart] - You may use a lightsaber ring. Use it wisely.
2025-08-08 21:45:05,914 INFO [Quickstart] - Sorry, you aren't allowed to drive the 'eagle5' winnebago!
2025-08-08 21:45:05,914 INFO [Quickstart] - You are permitted to 'drive' the tesla with license plate (id) 'tesla5'. Here are the keys - have fun!
itis42215221@itis-4221-5221:~/Desktop/shiro/samples/quickstart$
```

Figure 1.6: Execution Output After Role Modification

This figure shows the execution results of the Apache Shiro Quickstart application after modifying the shiro.ini configuration file to include the new 49sd role. The program was run using the Maven command:

mvn compile exec:java

- The logs confirm that user billchu successfully logged in with the credentials assigned in the updated configuration file.
- Upon authentication, Shiro evaluated the permissions for the 49sd role. The output confirms that:
 - billchu is not permitted to drive the Winnebago with license plate eagle5, as that permission was not assigned to the 49sd role.
 - billchu is permitted to drive any Tesla, including the Tesla with license plate tesla5, in line with the role configuration (tesla:drive:*)).

This output validates that the new 49sd role has been correctly applied and that Apache Shiro's role-based access control is functioning as expected, enforcing permissions exactly as defined in shiro.ini.

Conclusion

This assignment covered three distinct security improvements:

- Server-side request forgery was mitigated by validating URLs, enforcing allowlists, and disabling redirects before making outbound requests. SSRF can allow attackers to access internal services or local files[1], so robust input validation and network controls are essential.
- Insecure Direct Object Reference and access-control flaws were fixed by removing user-supplied identifiers from the URL, deriving the current user from session data, filtering database queries by user, and ensuring proper authorization checks[3]. Minimizing data exposure prevents information disclosure.
- The Apache Shiro configuration was extended to add a new role (49sd), assign it wildcard drive permissions on Teslas and Winnebagos, and associate it with user billchu. Updating the shiro.ini file and rerunning the quickstart example demonstrates that the new role functions as intended.

By following these remedial steps and referencing the OWASP recommendations on SSRF and IDOR, developers can harden the penetration_test application and adopt secure coding practices going forward.

References

1. Apache Shiro. (2025). *Apache Shiro 1.13 Documentation*. The Apache Software Foundation. Retrieved from <https://shiro.apache.org/documentation.html>
2. The Apache Software Foundation. (2025). *Quickstart Sample Project*. Retrieved from <https://shiro.apache.org/samples/quickstart.html>
3. Role-Based Access Control (RBAC). (2004). *ANSI INCITS 359-2004 Standard for Role Based Access Control*. American National Standards Institute. Retrieved from <https://csrc.nist.gov/projects/role-based-access-control>
4. Oracle. (2025). *Java Platform, Standard Edition Documentation*. Oracle Corporation. Retrieved from <https://docs.oracle.com/javase/8/docs/>
5. Apache Maven Project. (2025). *Maven Documentation*. The Apache Software Foundation. Retrieved from <https://maven.apache.org/guides/>
6. Spring Boot. (2025). *Spring Boot Reference Documentation*. Pivotal Software, Inc. Retrieved from <https://docs.spring.io/spring-boot/docs/current/reference/html/>